



杭州电子科技大学
HANGZHOU DIANZI UNIVERSITY

《数据结构课程设计》

实验报告

实验 2： 树与二叉树

姓 名： xx

学 号： xx

专 业： 网络空间安全

实验时间： 2024.11.14- 2024.11.28

指导老师： xx

杭州电子科技大学
网络空间安全学院

一、实验目的

- (1) 掌握二叉树的基本操作算法的实现与有关应用的实现;
- (2) 掌握线索二叉树的基本操作算法的实现;
- (3) 掌握赫夫曼树与赫夫曼编码的操作算法的实现;

二、实验内容与实验结果

实验 2-1 二叉树的实现与应用

1. 实验内容

I. 二叉树基本操作函数的实现

根据定义的二叉链表结构, 编写下列基本操作函数的 C/C++ 源代码:

- (1) `CreateBiTree(BiTree &T)`——根据先序遍历的字符序列, 创建一棵按二叉链表结构存储的二叉树, 指针变量 `T` 指向二叉树的根结点。
- (2) `PreOrderTraverse(BiTree T)`——递归先序遍历二叉树 `T`, 输出访问的结点字符序列;
- (3) `InOrderTraverse(BiTree T)`——递归中序遍历二叉树 `T`, 输出访问的结点字符序列;
- (4) `PostOrderTraverse(BiTree T)`——递归后序遍历二叉树 `T`, 输出访问的结点字符序列;

II. 应用函数的实现

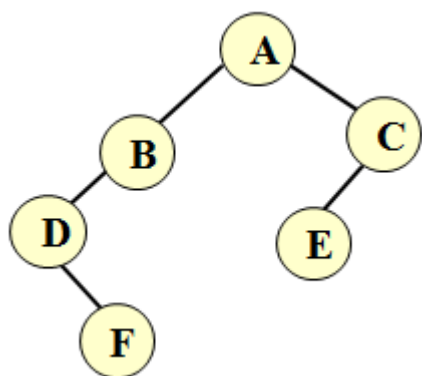
应用实例 1: 编一个函数 `TNodes(BiTree T, int d)`, 返回二叉树 `T` 度分别为 0,1,2 的结点数, 其中 `d` 为结点度数, 如 `TNodes(T,0)` 返回度为 0 (即叶子结点) 的结点数;

应用实例 2: 编一个函数 `High(BiTree T)`, 求二叉树 `T` 的高度;

应用实例 3: 编一个函数 `CreateBST(BiTree &T, const char *chars)`, 要求根据给定的字符序列 (字符不重复), 生成一棵二叉树。在二叉树中, 左子树所有结点的字符小于根结点的字符, 右子树所有结点的字符大于根结点的字符。左右子树也满足这个条件。

III. 编写一个主函数 `main()`, 检验上述操作函数是否正确, 实现以下操作:

- (1) 调用 `CreateBiTree(T)` 函数生成一棵二叉树 `T`。二叉树结构如下:



- (2) 分别调用先序遍历、中序遍历和后序遍历的递归函数输出相应的遍历结果;
- (3) 调用 `TNodes(T)` 函数, 输出二叉树 `T` 度分别为 0、1、2 的结点数。

(4) 调用 CreateBST(T1,"DBFCAEG"), CreateBST(T2,"ABCDEFGG"), 创建两棵二叉树, 对它们进行中序遍历, 并调用 High()函数输出其高度。分析比较其结果。

2. 源程序代码 (*.CPP)

(见附件 binarytree.cpp)

3. 运行结果

```
请输入先序遍历序列创建二叉树 (空结点用#表示):
```

```
ABD#F###CE###
```

```
先序遍历结果:
```

```
A B D F C E
```

```
中序遍历结果:
```

```
D F B A E C
```

```
后序遍历结果:
```

```
F D B E C A
```

```
度为0的结点数: 2
```

```
度为1的结点数: 3
```

```
度为2的结点数: 1
```

```
T1中序遍历:
```

```
A B C D E F G
```

```
T1的高度: 3
```

```
T2中序遍历:
```

```
A B C D E F G
```

```
T2的高度: 7
```

实验 2-2 线索二叉树的实现与应用

1. 实验内容

I. 线索二叉树基本操作与应用函数的实现

(1) InitBiThrTree(BiThrTree &T)——根据先序遍历的字符序列, 创建一棵按线索二叉链表结构存储的尚未线索化的二叉树, 在该二叉链表中, 若结点有左孩子 (即 lchild 非空), 则其 LTag=0, 否则 LTag=1, 若二叉树的结点有右孩子 (即 rchild 非空), 则其 RTag=0, 否则 RTag=1。指针变量 T 指向二叉树的根结点。

(2) InOrderThreading(BiThrTree &Thrt, BiThrTree T)——对 InitBiThrTree(BiThrTree &T)函数创建的二叉树 T 按中序遍历进行线索化, 线索化后的中序线索二叉树带“头结点”, 头结点的 data 域存放字符 '@', 指针 Thrt 指向该头结点, 头结点的左指针指向二叉树的根结点 (LTag=0), 右指针指向该二叉树中序遍历的最

后一个结点 (RTag=1)。同时, 该二叉树中序遍历第一个结点的左指针指向头结点 (LTag=1), 中序遍历最后一个结点的右指针也指向头结点 (RTag=1)。

线索化过程如下: 设 p 指向当前访问的结点, pre 指向 p 的前驱结点, 则:

(1) 对 p 所指的结点完成前驱线索化, 即: $p \rightarrow \text{LTag}=1$; $p \rightarrow \text{lchild}=\text{pre}$;

(2) 对 pre 所指的结点完成后继线索化, 即: $\text{pre} \rightarrow \text{RTag}=1$; $\text{pre} \rightarrow \text{rchild}=p$;

建议采用非递归的中序遍历方法实现, pre 的初始值为头结点指针 Thrt, 在中序遍历访问结点的位置 (printf()), 做以下操作:

```
if (!p->lchild) {p->LTag=1; p->lchild=pre;}
if (!pre->rchild) {pre->RTag=1; pre->rchild=p;}
pre=p;
```

在遍历结束后, 还要对最后一个结点线索化

```
pre->rchild=Thrt; pre->RTag=1;
Thrt->rchild=pre;
```

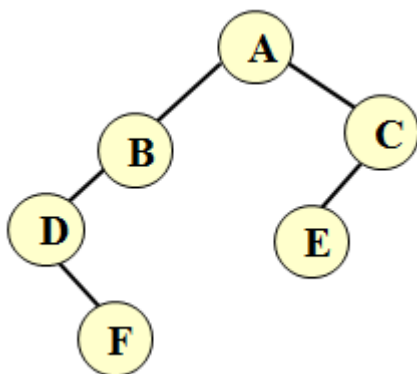
(3) InOrderTraverse(BiThrTree T)——按孩子中序遍历线索二叉树 (递归), 输出每个结点的数据, 格式如下:

| LTag | 左指针所指元素 | 本结点的值 | 右指针所指元素 | RTag |

如: | 1 | ^ | D | F | 0 | 或 | 1 | @ | D | F | 0 |

II. 编写一个主函数 main(), 检验上述操作函数是否正确, 实现以下操作:

(1) 调用 InitBiThrTree(T) 函数, 创建一棵按线索二叉链表结构存储的尚未线索化的二叉树; 二叉树结构如下:



(2) 调用 InOrderTraverse(T) 函数, 输出每个结点的数据。

(3) 调用 InOrderThreading(Thrt, T) 函数, 将 T 线索化成一棵中序线索二叉树;

(4) 调用 InOrderTraverse(Thrt) 函数, 输出每个结点的数据。

2. 源程序代码 (*.CPP)

(见附件 clue binarytree.cpp)

3. 运行结果

请输入二叉树的先序序列（用#表示空结点）： ABD#F###CE###
 尚未线索化的二叉树（中序遍历）：

1	^	D	F	0
1	^	F	^	1
0	D	B	^	1
0	B	A	C	0
1	^	E	^	1
0	E	C	^	1

线索化后的二叉树（中序遍历）：

1	@	D	F	0
1	D	F	B	1
0	D	B	A	1
0	B	A	C	0
1	A	E	C	1
0	E	C	@	1
0	A	@	C	1

实验 2-3 赫夫曼树与赫夫曼编码的实现

1. 实验内容

I. 赫夫曼树基本操作函数的实现

（1）InitHTree(HTree &HT, int *w, int n)——初始化赫夫曼树，其中 w 和 n 分别是权重数组和叶子结点数。要求根据定义的赫夫曼树结构，申请一个由 2n-1 个元素组成的一维数组 HT，HT[0..n-1]存放 n 个叶子结点的权重，其他元素和数据域的值置成-1。

（2）CreateHTree(HTree &HT, int n)——构造赫夫曼树 HT，其中 n 是叶子结点数。要求根据赫夫曼树的构造规则，生成一个由 2n-1 个结点组成的一棵赫夫曼树。

（3）HTCoding(HTree HT, HTCode &HC, int n)——生成赫夫曼编码 HC，其中 n 是叶子结点数。要求根据生成的赫夫曼树 HT，生成 n 个叶子结点的赫夫曼编码并输出。

II. 编写一个主函数 main()，检验上述操作函数是否正确，实现以下操作：

（1）设共有 8 个叶子结点，其权重 w={5, 29, 7, 8, 14, 23, 3, 11}，调用 InitHTree()函数，将叶子结点的权重存入赫夫曼树的存储结构 HT 中，并输出 HT 的内容；

（2）调用 CreateHTree()函数，构造一棵完整的赫夫曼树 HT，并输出 HT 的内容；

（3）调用 HTCoding()函数，输出赫夫曼树 HT 中各叶子结点的赫夫曼编码。

2. 源程序代码 (*.CPP)

（见附件 Huffman tree.cpp）

3. 运行结果

赫夫曼树初始化完成，所有结点信息如下：

结点 1: 权重 = 5, 双亲 = -1, 左孩子 = -1, 右孩子 = -1
结点 2: 权重 = 29, 双亲 = -1, 左孩子 = -1, 右孩子 = -1
结点 3: 权重 = 7, 双亲 = -1, 左孩子 = -1, 右孩子 = -1
结点 4: 权重 = 8, 双亲 = -1, 左孩子 = -1, 右孩子 = -1
结点 5: 权重 = 14, 双亲 = -1, 左孩子 = -1, 右孩子 = -1
结点 6: 权重 = 23, 双亲 = -1, 左孩子 = -1, 右孩子 = -1
结点 7: 权重 = 3, 双亲 = -1, 左孩子 = -1, 右孩子 = -1
结点 8: 权重 = 11, 双亲 = -1, 左孩子 = -1, 右孩子 = -1
结点 9: 权重 = -1, 双亲 = -1, 左孩子 = -1, 右孩子 = -1
结点 10: 权重 = -1, 双亲 = -1, 左孩子 = -1, 右孩子 = -1
结点 11: 权重 = -1, 双亲 = -1, 左孩子 = -1, 右孩子 = -1
结点 12: 权重 = -1, 双亲 = -1, 左孩子 = -1, 右孩子 = -1
结点 13: 权重 = -1, 双亲 = -1, 左孩子 = -1, 右孩子 = -1
结点 14: 权重 = -1, 双亲 = -1, 左孩子 = -1, 右孩子 = -1
结点 15: 权重 = -1, 双亲 = -1, 左孩子 = -1, 右孩子 = -1

赫夫曼树构造完成，所有结点信息如下：

结点 1: 权重 = 5, 双亲 = 9, 左孩子 = 0, 右孩子 = 0
结点 2: 权重 = 29, 双亲 = 14, 左孩子 = 0, 右孩子 = 0
结点 3: 权重 = 7, 双亲 = 10, 左孩子 = 0, 右孩子 = 0
结点 4: 权重 = 8, 双亲 = 10, 左孩子 = 0, 右孩子 = 0
结点 5: 权重 = 14, 双亲 = 12, 左孩子 = 0, 右孩子 = 0
结点 6: 权重 = 23, 双亲 = 13, 左孩子 = 0, 右孩子 = 0
结点 7: 权重 = 3, 双亲 = 9, 左孩子 = 0, 右孩子 = 0
结点 8: 权重 = 11, 双亲 = 11, 左孩子 = 0, 右孩子 = 0
结点 9: 权重 = 8, 双亲 = 11, 左孩子 = 7, 右孩子 = 1
结点 10: 权重 = 15, 双亲 = 12, 左孩子 = 3, 右孩子 = 4
结点 11: 权重 = 19, 双亲 = 13, 左孩子 = 9, 右孩子 = 8
结点 12: 权重 = 29, 双亲 = 14, 左孩子 = 5, 右孩子 = 10
结点 13: 权重 = 42, 双亲 = 15, 左孩子 = 11, 右孩子 = 6
结点 14: 权重 = 58, 双亲 = 15, 左孩子 = 2, 右孩子 = 12
结点 15: 权重 = 100, 双亲 = 0, 左孩子 = 13, 右孩子 = 14

赫夫曼编码生成完成：

叶子结点 1 (权重 = 5) : 编码 = 0001
叶子结点 2 (权重 = 29) : 编码 = 10
叶子结点 3 (权重 = 7) : 编码 = 1110
叶子结点 4 (权重 = 8) : 编码 = 1111
叶子结点 5 (权重 = 14) : 编码 = 110
叶子结点 6 (权重 = 23) : 编码 = 01
叶子结点 7 (权重 = 3) : 编码 = 0000
叶子结点 8 (权重 = 11) : 编码 = 001

三、实验小结

（通过实验得出的结论，对算法的功能以及与程序之间区别的理解、在编码是碰到的问题以及解决办法等心得以及得到的收获）

通过这几次实验，我对树结构及其相关算法有了更深入的理解。线索二叉树的实验让我掌握了如何通过前驱和后继的信息优化二叉树的存储和遍历方式，特别是线索化过程对指针的动态更新要求极高，初期实现时多次出现指针错误，但通过逐步调试和输出验证，成功解决了这些问题。实验还让我体会到递归函数逻辑清晰的重要性，分步骤分析和注释是实现正确代码的关键。

在赫夫曼树的实验中，通过权值最小结点的选择与合并，进一步理解了贪心算法的思想。这种构造方法简洁高效，但在实现中对动态数组管理和边界条件的处理提出了更高的要求。初期因数组初始化不足和下标越界导致错误，通过完善初始化和加强条件判断，最终实现了赫夫曼树的正确构造和编码生成。

总的来说，这些实验帮助我巩固了树的基本操作和算法思想，也提升了编程实现能力和调试能力。在实验中体会到，算法设计解决的是“如何做、怎么做”的问题，而程序实现则关注细节，特别是在动态内存分配、边界处理和递归操作中更需要严谨。通过实验，我不仅深化了对数据结构的理解，也积累了更多编码和问题解决的经验。